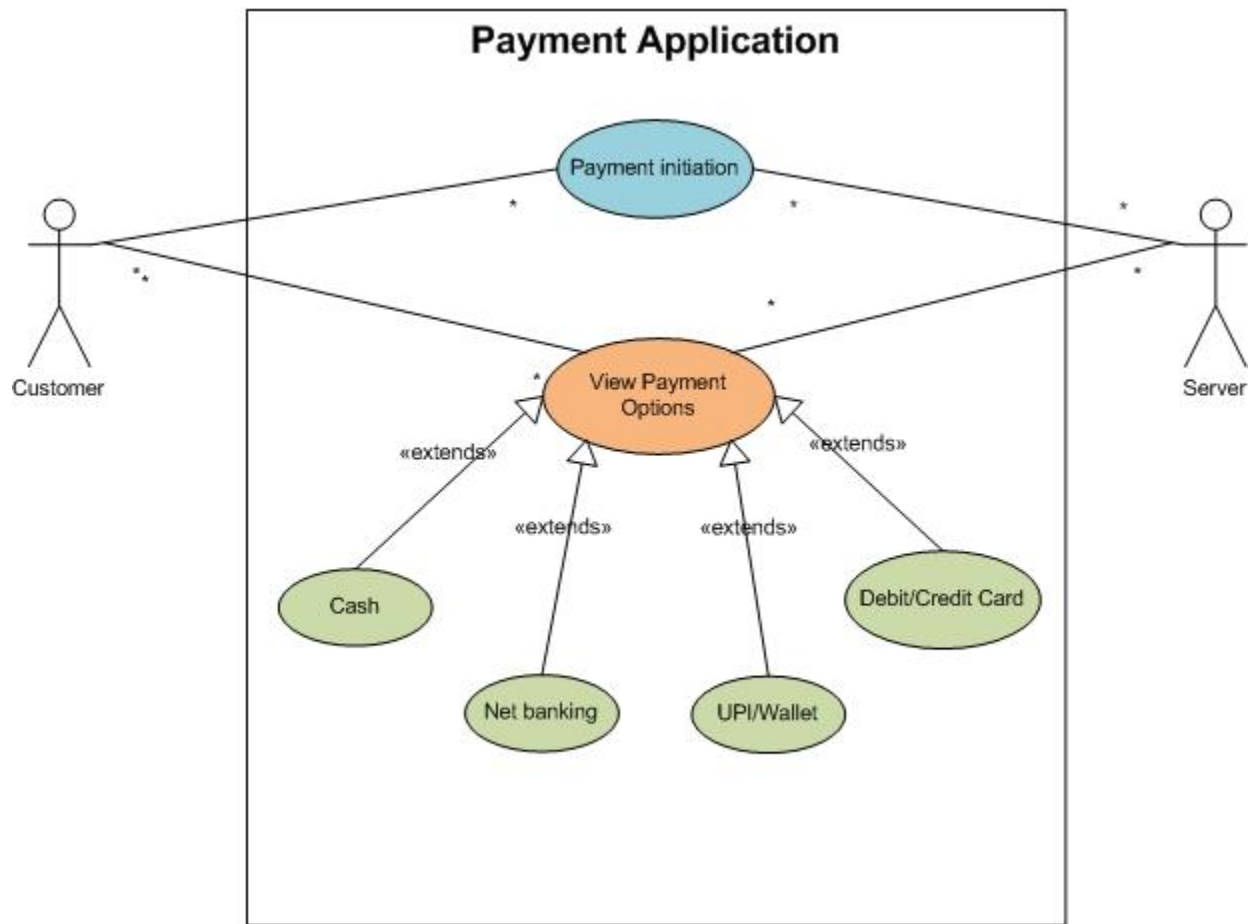


Omnichannel Payment Processing System

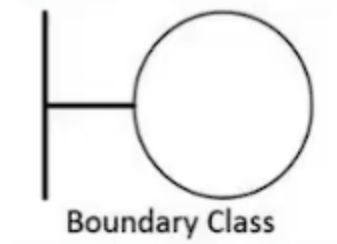
Q1. Draw a Use Case Diagram



Q2. Derive Boundary Classes, Controller classes, Entity Classes.

Boundary Classes

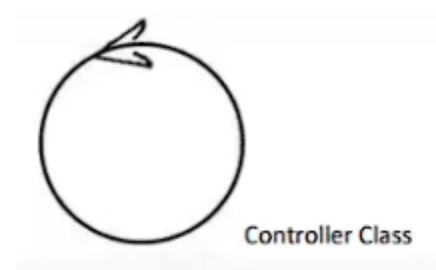
These handle system-external actor interactions, like UI screens or gateways for payment options.



- PaymentOptionBoundary: Manages selection of Card/Wallet/Cash/Net Banking.
- CardPaymentBoundary, WalletPaymentBoundary, CashPaymentBoundary, NetBankingPaymentBoundary: Specific input/validation interfaces to external providers.
- Role in Payment Flow: User/provider interfaces

Controller Classes

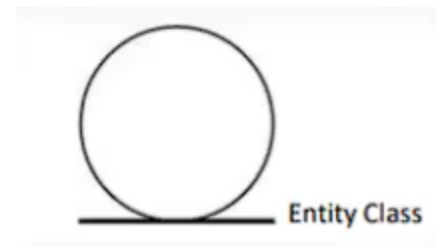
Controllers act as intermediaries, coordinating between boundaries and entities for business rules like validation and routing.



- PaymentController: Orchestrates overall payment flow, selects method, invokes validators.
- PaymentProcessorController: Handles specific processing (e.g., charge card, debit wallet).
- Role in Payment Flow: Logic orchestration

Entity Classes

These represent persistent core data and business logic, derived from nouns like Customer, Payment.



- Customer: ID, name, accountBalance.
- Payment: amount, method, timestamp, status.
- Transaction: transactionID, payment reference (1:1 with Payment).
- Role in Payment Flow: Data persistence

Q3. Place these classes on a three tier Architecture.

User/Presentation Tier

Hosts boundary classes for user/external interactions (user can see and click), ensuring loose coupling to back-end logic. With case study context the responsibilities include UI & external interfaces.

- PaymentOptionBoundary (selection UI).
- CardPaymentBoundary, WalletPaymentBoundary, CashPaymentBoundary, NetBankingPaymentBoundary (method-specific gateways/UIs).

Business Logic Tier

Contains controller classes to process rules, orchestrate flows, and mediate between tiers (decision making). With case study context the responsibilities include Rules & orchestration.

- PaymentController (routes payment method, validates).
- PaymentProcessorController (handles specifics like authorization).

Data Tier

Includes entity classes for core, persistent data storage and business objects (database storage). With case study context the responsibilities include Persistence & domain data.

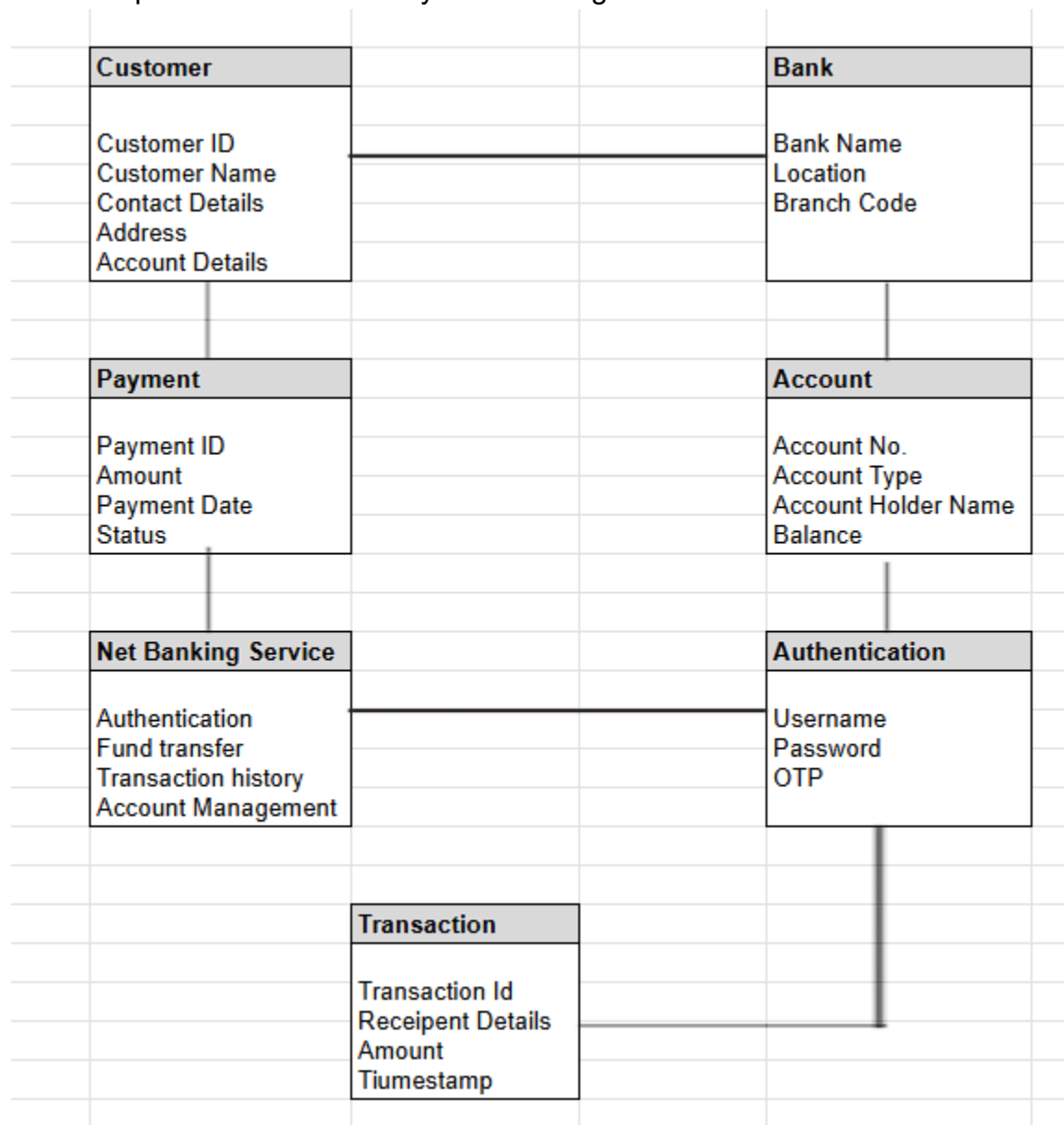
- Customer (ID, balance).
- Payment (amount, method, status).
- Transaction (ID, timestamp).

Users interact with boundaries → boundaries talk to controllers for decisions → controllers save data to entities

Q4. Explain Domain Model for Customer making payment through Net Banking.

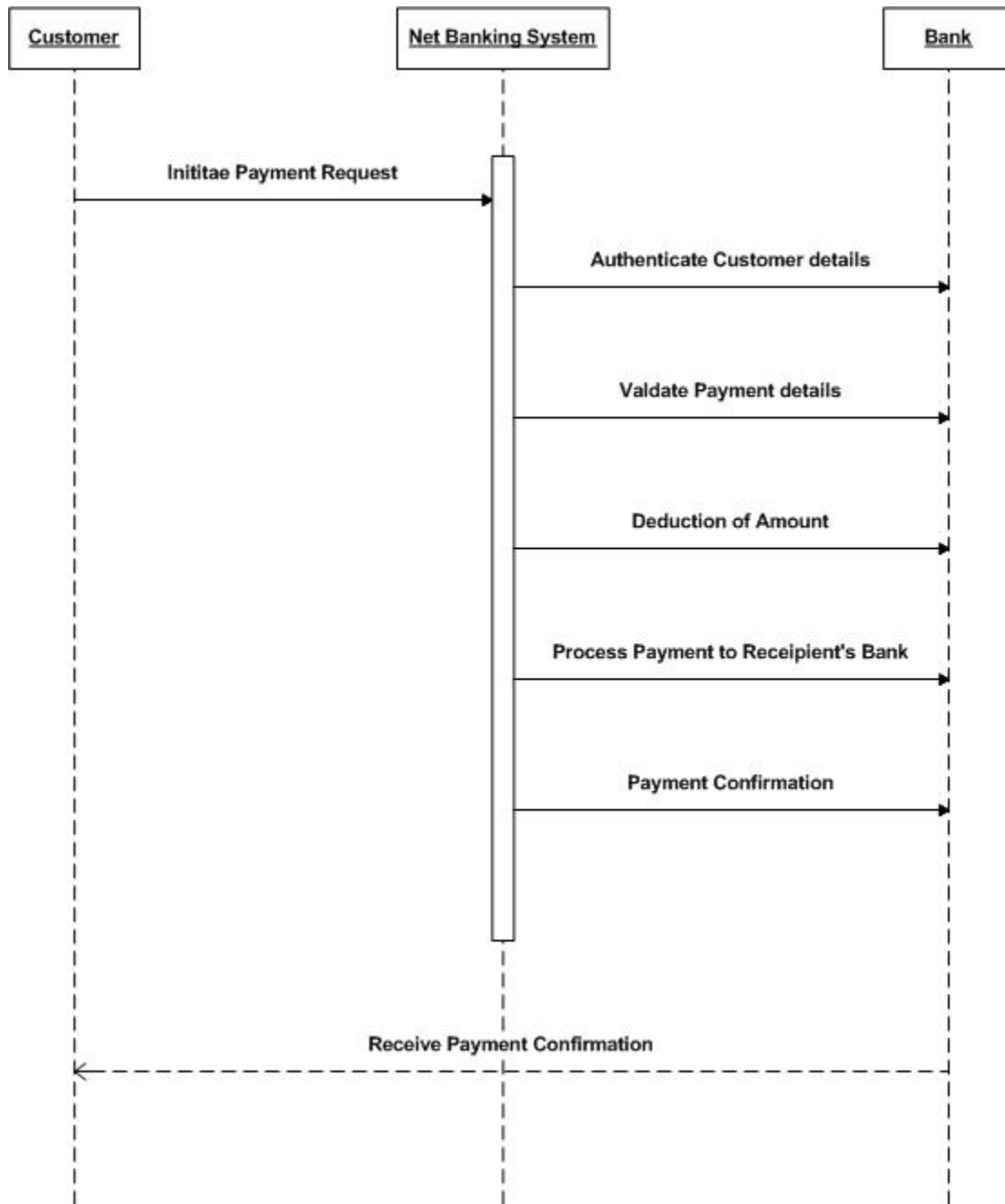
A domain model is a conceptual representation that defines the structure, relationships, and behaviours of entities within a specific problem domain.

The domain model for a Customer making payment through Net Banking is a UML class diagram illustrating real-world objects, attributes, associations, and behaviors specific to this payment method in the case study. It builds on the conceptual model by adding method-specific details for analysis and design handover.



Q5. Draw a sequence diagram for payment done by Customer Net Banking

A sequence diagram is a type of interaction diagram used in software engineering and system design to illustrate how process operate with one another and in what order.



Q6. Explain Conceptual Model for this Case

The conceptual model for the payment case study provides a high-level, implementation-independent view of the domain, focusing on key business entities, attributes, and relationships to aid stakeholder understanding and communication. It abstracts the scenario where a Customer makes payments via Card, Wallet, Cash, or Net Banking, without delving into technical classes like boundaries or controllers.

Key elements include:

Entities

Entities represent core business objects in the domain, independent of implementation.

- Customer: The person or account holder initiating payments via any method.
- PaymentMethod: The mechanism chosen for payment (Card, Wallet, Cash, Net Banking).
- Transaction: The record of a completed or attempted payment event.

Attributes

Attributes define essential properties of entities, capturing key business data for the payment domain.

- Customer:
 - customerID (unique identifier for tracking).
 - name (full name of the customer).
 - email (contact for receipts/confirmations).
 - phoneNumber (for OTP or support).
 - accountBalance (available funds to check sufficiency).
- PaymentMethod:
 - methodType (Card, Wallet, Cash, NetBanking to specify option).
 - details (cardNumber, walletID, bankAccount for method-specific info).
- Transaction:
 - transactionID (unique reference).
 - amount (payment value).
 - timestamp (date/time of action).
 - status (pending, success, failed for audit).

Relationships

Relationships show how entities connect, enforcing business rules like traceability.

- Customer (1) initiates multiple (*) PaymentMethod selections: One customer can choose many methods over time.
- Each PaymentMethod (*) produces one (1) Transaction: Each selection creates a single transaction record.
- Transaction belongs to one (1) Customer: Ensures ownership and history linking.